

Concept 8: Operational vs Historical Database (Polyglot Persistence)

Why This Matters

Your boss keeps coming back to this distinction because it's a fundamental architectural pattern in enterprise systems. A single database trying to serve both real-time operations AND historical analytics becomes a bottleneck. Understanding the split is key to designing a scalable platform.

The Two Roles a Database Can Play

Operational Database (OLTP — Online Transaction Processing)

Purpose: Serve the LIVE application. Handle the day-to-day create, read, update, delete operations.

Characteristics:

- Optimized for **fast individual transactions** (create an RFQ, update a stage, add a line item)
- Stores **current state** of data (active RFQs, pending reminders, current users)
- Needs **strong consistency** — when you update an RFQ status, the next read must reflect that change
- Schema is **normalized** (data split into related tables to avoid duplication)
- Queries are **simple and fast** (get RFQ by ID, list active RFQs, update a field)

In your project: The `rfq_manager_ms` database. It holds active RFQs, their current stages, line items, reminders. Every API call from the UI reads from or writes to this database.

Analogy: Your desk at work. It has the files you're currently working on. It's organized for quick access to active tasks.

Historical Database (OLAP — Online Analytical Processing)

Purpose: Store completed/archived data for analysis, intelligence, and institutional memory.

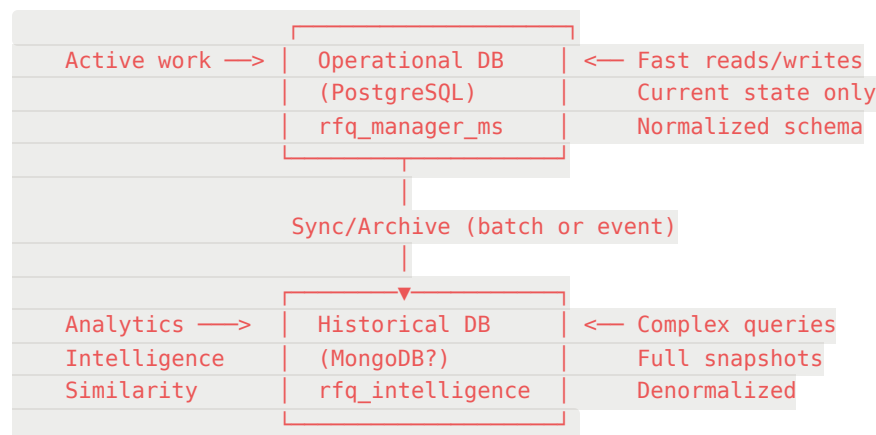
Characteristics:

- Optimized for **complex analytical queries** (find similar RFQs, average cost by material, win rate trends)
- Stores **historical snapshots** — what an RFQ looked like when it was completed, with all its context
- Can tolerate **eventual consistency** — a few minutes of delay in syncing data is acceptable
- Schema can be **denormalized** (data duplicated for faster reads — no need for joins)
- Queries are **complex and heavy** (aggregate across thousands of records, similarity search, trend analysis)

In your project: The `rfq_intelligence_ms` database. It holds completed RFQ snapshots for similarity search, cost estimation baselines, historical patterns. The chatbot queries this to find "similar past projects."

Analogy: Your company's filing cabinet / archive room. It has every completed project, organized for research and reference, not for daily work.

Why Separate Them?



Problem with one database for both:

1. **Performance conflict:** An analytics query scanning 10,000 historical RFQs slows down the API that's trying to load a single active RFQ for a user
2. **Schema conflict:** The operational schema is optimized for transactions (normalized, strict), while analytics benefits from denormalized, flexible structures
3. **Lifecycle conflict:** Active RFQs need tight validation; historical data needs to preserve what existed at the time, even if the schema has since changed
4. **Scaling conflict:** Operational needs consistent, low-latency access; analytics can be distributed and eventually consistent

Benefits of separation:

- **Performance isolation:** Heavy analytical queries don't impact the live application
- **Schema freedom:** The historical DB can store complete RFQ snapshots as documents, including data from external services, without being constrained by the operational schema
- **Independent scaling:** Scale the analytics DB horizontally for large datasets while keeping the operational DB lean
- **Technology freedom:** Use the BEST tool for each job (this is polyglot persistence)

Polyglot Persistence

Definition: Using different database technologies within the same system, each chosen for the specific needs of the service it supports.

This is NOT about complexity for complexity's sake. It's about recognizing that different services have fundamentally different data access patterns:

Service	Primary Need	Best Fit	Why
<code>rfq_manager_ms</code>	Fast CRUD, transactions,	PostgreSQL (SQL)	Relational data, ACID transactions, foreign key

Service	Primary Need	Best Fit	Why
	integrity		constraints
<code>rfq_intelligence_ms</code>	Flexible documents, similarity search, analytics	MongoDB + Vector DB	Schema flexibility for varied historical data, document-oriented queries
<code>rfq_chatbot_ms</code>	Conversation state, session management	Redis or PostgreSQL	Fast key-value access for sessions

How Data Flows Between Them

The historical database doesn't duplicate the operational one. Instead, when an RFQ is **completed** (or at regular intervals), a snapshot is created:

Option A — Event-based sync:

RFQ completed → event fired → intelligence service captures full snapshot → stores in historical DB

Option B — Batch sync:

Every night → batch identifies newly completed RFQs → creates snapshots → stores in historical DB

Option C — API-based sync:

RFQ completed → rfq_manager calls rfq_intelligence API → passes full RFQ data → intelligence stores it

The snapshot includes everything needed for future analysis: the RFQ details, all stages it went through, final cost, materials used, client info, documents referenced. It's a **self-contained record** that doesn't need to join back to the operational DB.

Connection to the MongoDB Discussion

This is exactly where your boss's suggestion of "trying MongoDB" fits architecturally:

- **Operational DB (rfq_manager):** PostgreSQL — because RFQ data is relational, needs ACID, and benefits from foreign key constraints. This is non-negotiable.

- **Historical DB (rfq_intelligence):** MongoDB is a strong candidate because:
 - Historical RFQ snapshots are **self-contained documents** (no joins needed)
 - Different RFQs from different time periods may have **different fields** (schema flexibility)
 - MongoDB's aggregation pipeline is well-suited for analytical queries
 - Combined with a vector database (for embeddings/similarity), it forms a powerful intelligence layer

This gives your boss what he wants (exploring MongoDB) while being **architecturally justified** — you're not using MongoDB because it's trendy, but because the historical/intelligence use case genuinely benefits from a document store.

Key Vocabulary

Term	Definition
OLTP	Online Transaction Processing — optimized for fast, individual transactions (operational)
OLAP	Online Analytical Processing — optimized for complex queries across large datasets (analytical)
Polyglot Persistence	Using different database technologies for different services based on their specific needs
Snapshot	A complete, self-contained copy of a record at a specific point in time
Normalization	Splitting data into separate related tables to eliminate duplication
Denormalization	Combining related data into a single structure for faster reads (accepting some duplication)
Sync	The process of copying/transferring data from one database to another